

Order Dependence in Saturated Cost Partitioning over Pattern Databases

Haniel Ulises

Automatic Discovery of Planning Heuristics

Abstract. Cost partitioning admits the sum of several admissible heuristics as an admissible heuristic, provided that the shares assigned to each action sum to at most the cost of that action. Saturated cost partitioning constructs those shares greedily: each component in turn retains the least cost function that preserves its own estimates, and the remainder is passed to the next component. The construction is admissible for every ordering of the components, but its accuracy is not, and this article reports a controlled measurement of that dependence within a verification framework in which exact goal distances are available by exhaustive enumeration. Three constructions over the same set of goal-pattern databases are compared against two admissible controls on Blocksworld instances of seven blocks, over the 263 958 reachable states for which h^* is computed exactly. A uniform partition is admissible and is the least accurate heuristic measured, at mean informedness 0.195 against 0.339 for the maximum over the same components. A single saturated ordering attains the same mean informedness as that maximum while expanding 50.8% more nodes under A^* , and a witness state is exhibited at which its estimate is strictly below the maximum. The maximum over the k cyclic rotations of the ordering is shown to dominate the maximum over the components at every state, and attains informedness 0.420 with 28.7% fewer expansions. A heterogeneous chain, in which the databases saturate and a landmark bound is priced with the residual cost function, dominates both controls by construction, and improves on the stronger of them by 1.9% of informedness. An accompanying analysis attributes the size of that margin to the distribution of cost mass: the databases retain between 70.4% and 78.6% of the total action cost and yield the better bound at 20.9% of the evaluated states.

Keywords. classical planning; admissible heuristics; cost partitioning; pattern databases; abstraction heuristics; landmarks.

1 Introduction

Optimal classical planning by A^* search [4] requires an admissible heuristic, and the accuracy of that heuristic determines the number of node expansions required. Several families of admissible heuristics are available, including delete relaxations [1], critical-path heuristics [5], landmark heuristics [8], and abstraction heuristics, of which pattern databases [2, 3, 6] are the oldest and the most widely implemented. Given several admissible heuristics, their pointwise maximum is admissible and is the standard way of combining them. The maximum, however, discards the information carried by every component that does not attain it at the state being evaluated.

Cost partitioning [9, 17, 11] provides a combination that can strictly exceed the maximum. Each component is evaluated under its own cost function over actions, and the sum of the components is admissible whenever the shares assigned to an action sum to at most the cost of that action in the task. The bound holds because no two components can charge for the same action twice. Saturated cost partitioning [12, 15] is a specific and inexpensive scheme for producing such shares. Each component in turn is assigned the least cost function under which it retains all of its own estimates, which for an abstraction heuristic is computable from the abstract transition relation, and the residual cost function is passed to the next component. The scheme is greedy and therefore depends on the order in which the components are considered.

This article reports a measurement of that dependence in a setting in which the ground truth is available. The verification harness of the framework enumerates the reachable state space of an instance and computes h^* at every state by backward Dijkstra search from the goal states, so that admissibility is decided and accuracy is measured directly instead of being inferred from search behaviour. Within that setting the following are reported.

1. A formal statement of the saturated cost function for a projection abstraction, with the treatment of transitions incident to abstract dead ends made explicit (Section 3), and a proof that a chain of such components spends the cost function of the task exactly once (Theorem 7).
2. A witness demonstrating that a single saturated ordering does not dominate the maximum over its own components (Theorem 9), together with the measured consequence: identical mean informedness and 50.8% more A^* expansions than the maximum.
3. A proof that the maximum over the k cyclic rotations of an ordering dominates the maximum over the components at every state (Theorem 11), and its measured effect.
4. A heterogeneous chain combining pattern databases with a landmark bound, shown to dominate both controls by construction (Theorem 14), and an analysis attributing the size of its empirical margin to the distribution of cost mass among the components (Section 7.3).

The constructions in items 1, 3 and 4 are not new; Section 9 attributes each to the literature. The contribution of this article is the independent implementation, the exact verification of every construction against enumerated goal distances, and the quantitative attribution of the observed margins.

2 Background

2.1 Planning tasks

A propositional STRIPS planning task is a tuple $\Pi = \langle P, A, s_0, G \rangle$, where P is a finite set of propositions, A a finite set of actions, $s_0 \subseteq P$ the initial state and $G \subseteq P$ the goal condition. A state is a subset $s \subseteq P$. An action $a = \langle \text{pre}(a), \text{add}(a), \text{del}(a) \rangle$ with $\text{pre}(a), \text{add}(a), \text{del}(a) \subseteq P$ is applicable in s if $\text{pre}(a) \subseteq s$, and yields the successor

$$s[a] = (s \setminus \text{del}(a)) \cup \text{add}(a).$$

A cost function is a map $c : A \rightarrow \mathbb{R}_{\geq 0}$. A state s is a goal state if $G \subseteq s$. A plan from s is a sequence of actions leading from s to a goal state, and $h_c^*(s)$ denotes the minimum cost of such a sequence under c , with $h_c^*(s) = \infty$ when no plan exists. A heuristic $h : 2^P \rightarrow \mathbb{R} \cup \{\infty\}$ is *admissible* for c if $h(s) \leq h_c^*(s)$ for every state s , and *consistent* if $h(s) \leq c(a) + h(s[a])$ for every s and every a applicable in s . Consistency together with $h(s) = 0$ at goal states implies admissibility. Where the cost function is that of the task it is omitted, and h^* abbreviates h_c^* .

2.2 Cost partitioning

Definition 1 (Cost partition). A tuple $\langle c_1, \dots, c_k \rangle$ of cost functions is a *cost partition* of c if $c_i(a) \geq 0$ for all i and a , and

$$\sum_{i=1}^k c_i(a) \leq c(a) \quad \text{for every } a \in A.$$

Cost may be left unspent; only overspending invalidates the bound. The following is the standard justification of the scheme and is stated here because every construction in Sections 3 to 5 is an instance of it.

Proposition 2 (Additivity under a cost partition). *Let $\langle c_1, \dots, c_k \rangle$ be a cost partition of c and let h_i be admissible for c_i for each i . Then $\sum_{i=1}^k h_i$ is admissible for c .*

Proof. Let s be a state with $h_c^*(s) < \infty$ and let $\langle a_1, \dots, a_m \rangle$ be a cost-optimal plan from s under c . The sequence is a plan under every cost function, so admissibility of h_i under c_i gives $h_i(s) \leq \sum_{j=1}^m c_i(a_j)$. Summing over i and exchanging the order of summation,

$$\sum_{i=1}^k h_i(s) \leq \sum_{j=1}^m \sum_{i=1}^k c_i(a_j) \leq \sum_{j=1}^m c(a_j) = h_c^*(s).$$

The inequality is vacuous where $h_c^*(s) = \infty$. $\square$

Proposition 2 places no requirement on the components beyond admissibility under their own shares. In particular the components need not be of the same family, a fact

exploited in Section 5, and the shares may be chosen as a function of the state, which is what makes optimal cost partitioning by linear programming [9, 10] well defined.

The following observation delimits the schemes that can be useful and motivates the general form.

Proposition 3 (Uniform scaling cannot exceed the maximum). *Let $h_1, \dots, h_k$ be admissible for c and let $w_1, \dots, w_k \geq 0$ with $\sum_i w_i \leq 1$. Let $h_i^{w_i}$ denote h_i evaluated under the cost function $w_i \cdot c$. Then $\sum_i h_i^{w_i}(s) \leq \max_j h_j(s)$ for every s .*

Proof. Scaling every action cost by w_i scales the goal distance, and hence the value of an abstraction or landmark bound built under it, by the same factor, so $h_i^{w_i} = w_i h_i$. Therefore $\sum_i w_i h_i(s) \leq (\sum_i w_i) \max_j h_j(s) \leq \max_j h_j(s)$. $\square$

Since $\max_j h_j$ is itself admissible, a partition of the shape covered by Proposition 3 is never preferable to the maximum. Only partitions in which components receive genuinely different cost functions over actions can exceed it.

2.3 Projection abstractions

Let $\Phi \subseteq P$ be a *pattern*. The projection onto Φ maps a state to $s \cap \Phi$ and an action a to

$$\pi(a) = \langle \text{pre}(a) \cap \Phi, \text{add}(a) \cap \Phi, \text{del}(a) \cap \Phi \rangle.$$

The projection is a homomorphism of the transition system, since intersection distributes over the STRIPS transition and a projected precondition is weaker than the concrete one. Every concrete plan therefore induces an abstract plan of the same cost, the abstract goal distance is a lower bound on the concrete one, and the resulting heuristic is admissible and consistent. A *pattern database* stores the abstract goal distance of every abstract state and evaluates by masking and lookup [2, 3]. Actions with $\text{add}(a) \cap \Phi = \text{del}(a) \cap \Phi = \emptyset$ induce self-loops in the abstraction and are discarded; write A_Φ for the set of actions retained.

Patterns here are constructed by bounded causal closure: each goal condition seeds one pattern, the preconditions of the actions achieving the seed are added, then the preconditions of the actions achieving those, up to a budget of twelve propositions. Pattern selection is a research problem in its own right [6, 14] and no claim is made for this procedure beyond determinism and adequacy as a source of components.

3 Saturated cost partitioning

3.1 The saturated cost function

A pattern database stores goal distances computed under one cost function. If the stored table is to be read while a smaller cost function is charged for, the table must remain a bound under that smaller function.

Definition 4 (Saturated cost function). Let D_i be a pattern database over pattern Φ_i , built under a cost function d ,

with abstract goal distances h_i . Its *saturated cost function* $\text{scf}_i : A \rightarrow \mathbb{R}_{\geq 0}$ is

$$\begin{aligned} \text{scf}_i(a) &= \min(d(a), \delta_i(a)), \\ \delta_i(a) &= \max(\{0\} \cup \Delta_i(a)), \\ \Delta_i(a) &= \{ h_i(u) - h_i(v) \mid \\ &\quad u \xrightarrow{\pi(a)} v, h_i(u), h_i(v) < \infty \}, \end{aligned}$$

where the transitions range over the abstract transition relation and $\text{scf}_i(a) = 0$ for every $a \notin A_\Phi$.

Two families of transition are excluded from $\Delta_i(a)$ and the exclusions are not cosmetic. A transition with $h_i(u) = \infty$ leaves an abstract dead end, which no search expands, and the difference is undefined. A transition with $h_i(v) = \infty$ enters an abstract dead end, so that $h_i(u) - h_i(v)$ is unbounded below and the constraint it imposes is satisfied by every non-negative value. Charging for either transition would remove cost from subsequent components without improving the present one.

Theorem 5 (Saturation preserves the table). *h_i is admissible for the projected task under scf_i .*

Proof. Every abstract goal state satisfies $h_i = 0$. For every abstract transition $u \xrightarrow{\pi(a)} v$ with $h_i(u), h_i(v) < \infty$, Definition 4 gives $\text{scf}_i(a) \geq h_i(u) - h_i(v)$, hence $h_i(u) \leq \text{scf}_i(a) + h_i(v)$; where $h_i(v) = \infty$ the inequality is immediate. Thus h_i is consistent and goal aware under scf_i .

Let u_0 be an abstract state with $h_{\text{scf}_i}^*(u_0) < \infty$ and let $\langle a_1, \dots, a_m \rangle$ be a cost-optimal plan under scf_i through $u_0, \dots, u_m$ with u_m an abstract goal state. Every u_j lies on a path to a goal state, so $h_i(u_j) < \infty$ for all j . Telescoping the consistency inequality along the path,

$$h_i(u_0) \leq \sum_{j=1}^m \text{scf}_i(a_j) + h_i(u_m) = h_{\text{scf}_i}^*(u_0). \quad \square$$

The clipping at $d(a)$ in Definition 4 is what makes scf_i a share of d and not a demand upon it. A drop exceeding $d(a)$ is possible, because the projection may identify several concrete actions with one abstract action; retaining the unclipped drop would permit a component to charge more for an action than the task ever pays for it.

3.2 Chains

Definition 6 (Saturated chain). Given an ordered sequence of patterns $\Phi_1, \dots, \Phi_k$, set $r_0 = c$ and, for $i = 1, \dots, k$: build D_i under r_{i-1} , let $c_i = \text{scf}_i$ computed under r_{i-1} , and set $r_i = r_{i-1} - c_i$. The heuristic is $h_{\text{scp}} = \sum_{i=1}^k h_i$, and r_k is the *residual* cost function of the chain.

Theorem 7 (The chain spends the cost function exactly once). *For every action a : $c_i(a) \geq 0$ for all i , $r_k(a) \geq 0$, and $\sum_{i=1}^k c_i(a) + r_k(a) = c(a)$. In particular $\langle c_1, \dots, c_k \rangle$ and $\langle c_1, \dots, c_k, r_k \rangle$ are cost partitions of c .*

Proof. By induction on i . Definition 4 clips c_i at the cost function under which D_i was built, so $0 \leq c_i(a) \leq r_{i-1}(a)$, and therefore $r_i(a) = r_{i-1}(a) - c_i(a)$ is non-negative and the subtraction is exact. Telescoping, $\sum_{i \leq k} c_i(a) = r_0(a) - r_k(a) = c(a) - r_k(a)$. $\square$

Algorithm 1 Saturated chain over an ordered pattern sequence

Require: task Π with cost function c ; patterns $\Phi_1, \dots, \Phi_k$
Ensure: databases $D_1, \dots, D_k$; shares $c_1, \dots, c_k$; residual r

```

1:  $r \leftarrow c$ 
2: for  $i \leftarrow 1$  to  $k$  do
3:    $D_i \leftarrow \text{BUILD\_DATABASE}(\Pi, \Phi_i, r)$ 
4:    $c_i \leftarrow \text{SATURATE}(D_i, r)$  ▷ Definition 4
5:    $r \leftarrow r - c_i$ 
6: end for

```

Corollary 8. *h_{scp} is admissible and consistent for Π under c .*

Proof. Admissibility follows from Theorems 5 and 7 with Proposition 2. Each h_i is an abstraction heuristic and is consistent under c_i ; a sum of functions consistent under the members of a cost partition is consistent under c , since summing the individual consistency inequalities yields $\sum_i h_i(s) \leq \sum_i c_i(a) + \sum_i h_i(s \llbracket a \rrbracket) \leq c(a) + \sum_i h_i(s \llbracket a \rrbracket)$. $\square$

4 Order dependence

Corollary 8 holds for every ordering of the patterns. Accuracy does not, and the failure is not confined to contrived tasks.

Theorem 9 (A saturated chain does not dominate the maximum). *There exist a task Π , a set of patterns and a state s such that $h_{\text{scp}}(s) < \max_i h_i^c(s)$, where h_i^c denotes the database over Φ_i built under the cost function of the task.*

Proof. Let $P = \{\text{start}, \text{left}, \text{right}\}$, $s_0 = \{\text{start}\}$, $G = \{\text{left}, \text{right}\}$, and let A consist of three unit-cost actions: **both**, with precondition $\{\text{start}\}$ and add effect $\{\text{left}, \text{right}\}$; **take_left** and **take_right**, each with precondition $\{\text{start}\}$ and add effect the corresponding goal proposition. Bounded causal closure from each goal condition yields $\Phi_1 = \{\text{start}, \text{left}\}$ and $\Phi_2 = \{\text{start}, \text{right}\}$.

Consider $s = \{\text{start}, \text{left}\}$, at which $h^*(s) = 1$. Under c , the database over Φ_1 reports $h_1^c(s) = 0$, since its abstract goal $\{\text{left}\}$ is already satisfied, and the database over Φ_2 reports $h_2^c(s) = 1$. The maximum over the components is therefore 1.

Take the ordering $\langle \Phi_1, \Phi_2 \rangle$. The abstraction of Φ_1 contains the transition $\{\text{start}\} \xrightarrow{\text{both}} \{\text{start}, \text{left}\}$, along which h_1 decreases from 1 to 0, so $\text{scf}_1(\text{both}) = 1$ and $r_1(\text{both}) = 0$. The database over Φ_2 is then built with **both** free of charge and reaches its abstract goal from $\{\text{start}\}$ at cost 0, giving $h_2(s) = 0$. Hence $h_{\text{scp}}(s) = h_1(s) + h_2(s) = 0 < 1$. $\square$

Both values are admissible; only one is informative. The mechanism is general: a component that has already reached its own abstract goal at s nonetheless saturates against the transitions leading into that goal, and the cost it retains is precisely the cost the next component required.

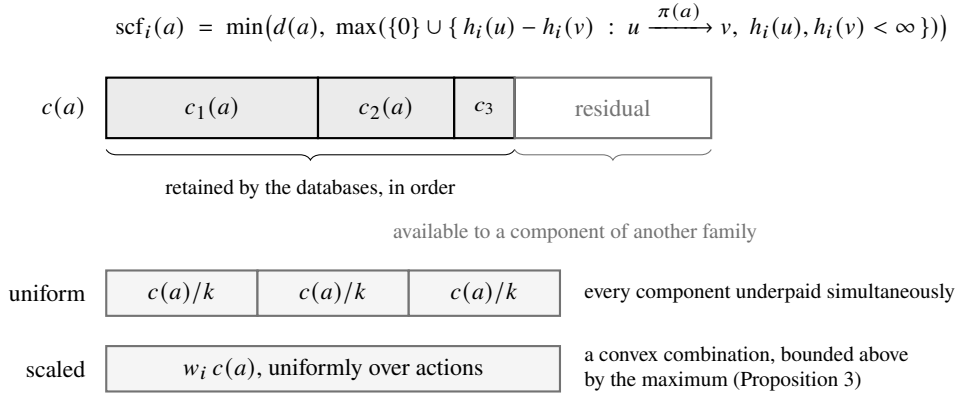


Figure 1. The cost of a single action under three partitioning schemes. All three are admissible. The saturated chain assigns each component the least cost function preserving its own estimates and passes the residual on; the uniform scheme divides the cost without reference to what any component can use; the scaled scheme is a convex combination and is dominated by the pointwise maximum of its own components.

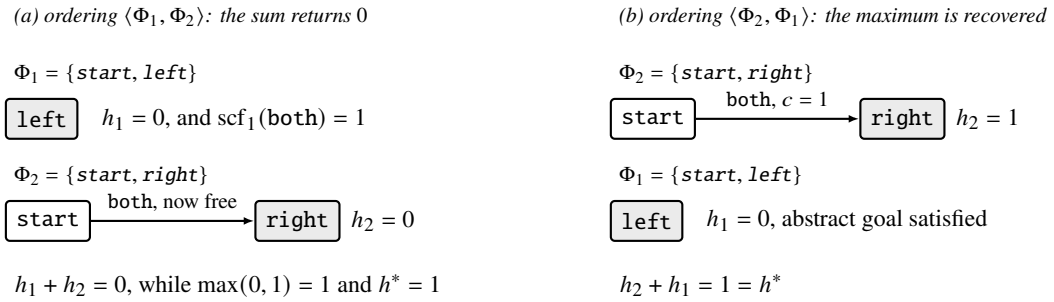


Figure 2. The witness of Theorem 9, evaluated at $s = \{\text{start}, \text{left}\}$, and the ordering that recovers the maximum. A component already at its abstract goal saturates against the transitions entering that goal, and the cost it retains is the cost the subsequent component required. Theorem 11 generalises the repair.

Definition 10 (Rotation family). For patterns $\Phi_1, \dots, \Phi_k$ let σ_i denote the cyclic rotation beginning at Φ_i , and let $h^{(i)}$ be the saturated chain under σ_i . The *rotation family* heuristic is $h_{\text{rot}} = \max_{1 \leq i \leq k} h^{(i)}$.

Theorem 11 (Rotations dominate the maximum). h_{rot} is admissible, and $h_{\text{rot}}(s) \geq \max_{1 \leq i \leq k} h_i^c(s)$ at every state s .

Proof. Admissibility follows from Corollary 8 applied to each rotation and from the fact that a pointwise maximum of admissible heuristics is admissible. For the domination, fix i . In σ_i the component Φ_i is first, hence is built under $r_0 = c$, and its contribution to $h^{(i)}$ is exactly $h_i^c(s)$. Every remaining summand of $h^{(i)}$ is an abstract goal distance and is therefore non-negative, so $h^{(i)}(s) \geq h_i^c(s)$. Taking the maximum over i on both sides gives the claim. $\square$

Remark 12. The rotation family is minimal among order families with this property in the following sense: removing σ_i removes the only ordering in which Φ_i is unpriced by any other component, and the argument above no longer covers a state at which Φ_i attains the maximum. It is not the only such family; any set of permutations in which every component appears first at least once suffices, and the rotations are the smallest such set. Selecting a smaller and better set is a search over permutations and is discussed in Section 8.

5 Heterogeneous partitions

Proposition 2 does not require the components to belong to one family. The two admissible components available in this framework have complementary profiles: the databases are consistent, evaluate in constant time, and capture inter-actions internal to a pattern; the landmark bound is better informed, is inconsistent, and requires one relaxed reachability fixpoint per candidate proposition at each evaluation. The landmark bound used here is the uniform cost partition over the landmarks of a state [8]: each action divides its cost equally among the landmarks it may achieve, and each landmark contributes the least share assigned to it by any of its achievers.

A landmark bound of this kind admits no saturated cost function in the sense of Definition 4. An abstraction stores a function over the whole abstract state space, so the least cost function preserving it is a property of the stored table; the landmarks of a state are recomputed at that state, and there is no stored function to preserve. The analogue would be a distinct cost function at each state, for which the databases cannot be rebuilt. The chain therefore admits only one arrangement: the databases saturate, and the landmark bound is priced with the residual r_k , which requires no saturation because no component follows it.

Definition 13 (Mixed family). For each rotation σ_i let $h^{(i)}$ be its saturated chain over the databases and $r^{(i)}$ its residual. Writing h_{LM}^d for the landmark bound priced by cost function

d ,

$$h_{\text{mix}}(s) = \max \left(h_{\text{LM}}^c(s), \max_{1 \leq i \leq k} [h^{(i)}(s) + h_{\text{LM}}^{r(i)}(s)] \right).$$

Theorem 14 (Domination of both controls). h_{mix} is admissible, and for every state s , $h_{\text{mix}}(s) \geq \max_i h_i^c(s)$ and $h_{\text{mix}}(s) \geq h_{\text{LM}}^c(s)$.

Proof. By Theorem 7 the tuple $\langle c_1^{(i)}, \dots, c_k^{(i)}, r^{(i)} \rangle$ is a cost partition of c , so each bracketed sum is admissible by Proposition 2. The first argument of the outer maximum is the degenerate partition assigning the entire cost function to the landmark bound, which is admissible. A pointwise maximum of admissible heuristics is admissible.

The landmark control is dominated because it is a member of the family. The abstraction control is dominated by the argument of Theorem 11, since the i -th bracket contains $h^{(i)}(s) \geq h_i^c(s)$ together with a non-negative landmark term. $\square$

The degenerate member is required only for the second half of the proof, and costs one additional pricing of a landmark set that has already been generated. Since the landmark set of a state is determined by relaxed reachability and is independent of the cost function, generation is performed once per state and the set is priced $k + 1$ times.

Remark 15. h_{mix} inherits the inconsistency of the landmark bound. A^* with an inconsistent heuristic may reopen closed states, and Table 1 reports the frequency; optimality is preserved by reopening.

6 Implementation

All constructions are header-only additions to an existing C++20 planning framework in which cost functions were already first-class: landmark generation, database construction and exact goal-distance computation each accept a supplied cost function, and a predicate deciding Definition 1 was already available. Three changes were required before Definition 4 could be implemented.

Recovering concrete action indices. A projection discards the actions of $A \setminus A_\Phi$, so the action indices of the abstract task are not those of the concrete task. A saturated cost function is computed over abstract transitions and must be expressed over A , so the projection now records the concrete origin of each retained action.

Naming the retention predicate. The test $\text{add}(a) \cap \Phi \neq \emptyset \vee \text{del}(a) \cap \Phi \neq \emptyset$ was previously evaluated inside the projection loop. The uniform scheme requires the answer before any projection exists, in order to count the components able to use each action, so the predicate is now a named function that the projection calls.

Separating landmark generation from pricing. Landmark generation is independent of the cost function and is the expensive half of the landmark bound. Pricing is now a separate entry point taking a cost function, so one factory

serves the $k + 1$ shares required by Definition 13 at the cost of one generation per state.

The saturated cost function is computed in a single pass over the abstract transition relation at construction time, so building a database is asymptotically no more expensive than before. The unit test suite was extended from 87 to 104 tests. The additions verify that each saturated share is bounded by the cost function it was derived from, that share and residual sum to that function exactly (Theorem 7), that each database remains admissible when verified against goal distances recomputed under its own saturated costs (Theorem 5), that every construction yields a cost partition (Definition 1), that the witness of Theorem 9 holds as an exact pair of values, and that the domination of Theorem 11 holds at every state of three fixtures.

7 Empirical evaluation

7.1 Experimental setup

The benchmark comprises the four Blocksworld [16] instances of seven blocks used as the verification set of the framework. Each has 98 actions and 7 goal conditions, hence yields 7 patterns under a budget of twelve propositions, so the rotation and mixed families build 49 databases per instance. The reachable state spaces total 263 958 states, of which 263 954 have a finite non-zero goal distance.

Two measurements are reported. *Informedness* is the mean of h/h^* over states with finite non-zero h^* , computed by enumerating the reachable state space and running backward Dijkstra search from the goal states; it requires no search and is deterministic. *Expansions* are the totals over the four instances of A^* run to optimality, with tie-breaking in the open list by insertion order. Every configuration returns optimal plans of total cost 60.

Wall-clock time is separated into *setup*, the interval preceding the first node expansion, which for the abstraction heuristics is the enumeration of every projection, and *search*. The separation is necessary because the rotation and mixed families build k chains of k databases, and it is that component of the cost which grows. All measurements were taken with g++ 11.4.0 at C++20 in a release build on a single core.

7.2 Results

Table 1 reports the comparison. Four observations follow.

A legal partition is not by itself valuable. The uniform partition satisfies Definition 1 in its general form and is the least accurate heuristic measured, at informedness 0.195 against 0.339 for the maximum over the same components, with 3.75 times as many expansions. Dividing the cost of an action among the components able to use it, without reference to what any of them can spend, underpays all of them simultaneously.

A mean conceals a bound that collapses. The single saturated ordering and the maximum report identical mean informedness to three decimal places, yet the former expands 50.8% more nodes. The minimum column identifies the

Table 1. The four seven-block Blocksworld instances, verified over the same 263 958 enumerated states and searched with A^* . Informedness and its minimum are obtained from exact goal distances without search; expansions, reopenings and search time from the planner. *Lower g* counts states rediscovered along a cheaper path. Every configuration returns optimal plans of total cost 60. The two rows marked † are the admissible controls.

Heuristic	Informedness	Min h/h^*	Expanded	Lower g	Search	Setup	Properties
goal_count	0.367	0.045	15 451	0	0.01 s	0.00 s	admissible, consistent
relaxed_layers	0.344	0.143	28 689	0	0.15 s	0.01 s	admissible, consistent
† pdb, maximum over components	0.339	0.111	18 341	882	0.03 s	0.24 s	admissible, consistent
pdb_uniform, uniform partition	0.195	0.026	68 781	0	0.08 s	0.25 s	admissible, consistent
scp, one saturated ordering	0.339	0.000	27 655	2 037	0.03 s	0.22 s	admissible, consistent
scp_rotations, rotation family	0.420	0.167	13 084	379	0.09 s	1.71 s	admissible, consistent
† landmark_cost	0.520	0.268	4 166	378	0.48 s	0.00 s	admissible, inconsistent
scp_mixed, mixed family	0.530	0.268	4 067	324	1.06 s	1.64 s	admissible, inconsistent

Table 2. A^* expansions per instance. The ordering of the heuristics is identical on all four instances.

Heuristic	07-00	07-01	07-02	07-03
goal_count	7 325	182	7 364	580
relaxed_layers	12 421	664	13 204	2 400
pdb	7 895	761	8 626	1 059
pdb_uniform	29 686	2 223	29 213	7 659
scp	10 851	1 057	12 954	2 793
scp_rotations	5 784	271	6 540	489
landmark_cost	2 518	53	1 506	89
scp_mixed	2 465	51	1 463	88

cause: at 0.000, the mechanism of Theorem 9 is present at scale. A mean over states conceals a bound that falls to zero at some of them, and A^* does not, because those states are precisely the ones it must then expand.

The rotation family improves accuracy and increases wall-clock time at both ends. Informedness improves by 23.9% over the maximum and expansions fall by 28.7%, with the minimum rising from 0.111 to 0.167, which is the floor guaranteed by Theorem 11. Setup rises from 0.24 s to 1.71 s, a factor of approximately k , and search rises from 0.03 s to 0.09 s despite the reduction in expansions, because a single evaluation requires $k^2 = 49$ table lookups instead of $k = 7$. At seven blocks the entire search occupies a tenth of a second, so no amortisation occurs; the construction is a loss in wall-clock time and a gain in the quantity the framework optimises.

The mixed family dominates both controls, with a small margin. Informedness is 0.530 against 0.520 and expansions 4 067 against 4 166. Theorem 14 guarantees the sign of both differences; their magnitude is 1.9% and 2.4% respectively. Search time doubles, since each evaluation prices the landmark set $k + 1$ times in addition to performing k^2 table lookups.

Table 2 shows that the aggregate figures are not carried by any single instance. The single saturated ordering is worse than the maximum on all four instances and the rotation family is better on all four.

Table 3. Which member of the mixed family attains the maximum, over the 263 954 states with finite non-zero goal distance.

Attaining member	States	Share
Landmark bound alone, on the whole c	199 434	75.6 %
A mixed rotation, databases and residual	55 151	20.9 %
Tie	9 369	3.5 %
Mean value, landmark bound alone		8.182
Mean value, best mixed rotation		6.631

7.3 Distribution of cost mass

The margin obtained by the mixed family admits a direct explanation that requires no search. At every state with finite non-zero goal distance, the two arguments of the outer maximum of Definition 13 were compared.

Table 3 reports the outcome. The saturated databases retain between 70.4% and 78.6% of the total action cost of an instance, depending on the instance, and yield the better bound at 20.9% of the evaluated states. At approximately three states in four the preferable allocation assigns the databases nothing, and h_{mix} realises that allocation only because the degenerate member is present in the family.

This is a statement about the components and not about the scheme. Saturation is optimal for a single component considered in isolation, since it retains the least cost function preserving that component’s estimates. It is not optimal for a chain, because the cost required to preserve a component’s table is not the cost required for that component to be informative: a component that has reached its abstract goal preserves a table of zeros at full price. The seven goal-condition patterns of a Blocksworld instance overlap substantially, and each preserves a quantity of cost that the landmark bound would have priced more productively.

8 Discussion and limitations

No upper reference point is reported. Optimal cost partitioning, computed per state by a linear program over the shares [9, 10], is the standard reference against which the recovery of a partitioning scheme is measured. It is not implemented here, so the values 0.420 and 0.530 are comparisons against the controls and not against the bound attainable from these components. Establishing that reference is the first item of further work.

A single domain and small instances. All measurements are on Blocksworld, and seven blocks is the largest size at which exhaustive enumeration remains inexpensive. Both constructions are exponential in the pattern size and the rotation family is quadratic in the number of goal conditions, so the reported setup times are lower bounds on what a larger domain would require. In particular, the claim that the databases retain roughly three quarters of the cost mass is a claim about this domain and these patterns, and a domain whose goal conditions interact less would be expected to behave differently.

Pattern selection is not addressed. One pattern per goal condition is generated by bounded causal closure to twelve propositions. Raising that budget continues to improve informedness at seven blocks, which indicates that pattern selection influences the result at least as strongly as the partition over the patterns. A scheme that allocates cost well among poorly selected components remains limited by those components.

The rotation family is exhaustive, not selective. Theorem 11 obtains its guarantee by evaluating all k orderings. A single well-chosen ordering would cost a k -th of the setup and would forfeit the guarantee. Nothing established here indicates how to select it, and Remark 12 together with Section 7.3 suggests that the ordering, and not the saturation rule, is where the remaining accuracy resides. An ordering is a permutation of the components: a small, interpretable object, and one for which the objective is computable without running a search.

9 Related work

Pattern databases were introduced for the sliding-tile puzzle by Culberson and Schaeffer [2] and brought to domain-independent planning by Edelkamp [3], with automatic pattern selection developed by Haslum et al. [6]. Additivity of abstraction heuristics under disjoint action costs was formalised by Yang et al. [17], and the general form of cost partitioning, together with its optimal computation by linear programming, is due to Katz and Domshlak [9]; the extension to general, possibly negative, operator cost partitioning is due to Pommerening et al. [11]. The uniform cost partition over landmarks used as the second component here is that of Karpas and Domshlak [8], and the relationship between landmark and abstraction heuristics under cost partitioning was examined by Helmert and Domshlak [7].

Saturated cost partitioning is due to Seipp and Helmert [12], who introduced it for Cartesian abstractions, and is developed at length by Seipp, Keller and Helmert [15]. Its dependence on the ordering of the components, and the use of several orderings selected for diversity, are treated in the same line of work [13, 15]; the rotation family of Definition 10 is a specific deterministic instance of that idea, chosen here for the guarantee of Theorem 11 rather than for diversity. Combining components of different families under one saturated partition is likewise established practice in that literature. Accordingly, no claim of novelty is made for the constructions themselves. What this article

contributes is an independent implementation inside a verification framework where h^* is available exactly, the resulting confirmation of admissibility and consistency by exhaustive enumeration rather than by argument alone, and the quantitative attribution in Section 7.3 of the margin obtained by a heterogeneous partition to the distribution of cost mass among its components.

10 Conclusion

Three cost partitioning schemes over the same set of goal-pattern databases were implemented and verified against exact goal distances. A uniform partition is admissible and is the least accurate heuristic measured. A single saturated ordering is admissible, attains the mean informedness of the maximum over its own components, and expands substantially more nodes, because it does not dominate that maximum; a witness state was exhibited and the mechanism identified. The maximum over the cyclic rotations of the ordering was proved to dominate the maximum over the components at every state and improves informedness by 23.9% and expansions by 28.7%. A heterogeneous chain combining the databases with a landmark bound dominates both admissible controls by construction and improves on the stronger of them by 1.9% of informedness.

The size of that final margin was traced to the distribution of cost mass: the databases retain roughly three quarters of the total action cost while yielding the better bound at one state in five. Saturation is optimal for an isolated component and is not optimal for a chain, and the discrepancy is governed by the ordering. Establishing an optimal cost partitioning reference point, and searching the space of orderings rather than enumerating a family of them, follow directly.

Reproducibility

The implementation, the benchmark instances and the measurement scripts are contained in the `cost-partitioning` branch of the project repository. Both command-line tools emit a single self-describing JSON record carrying the build provenance, the resource limits and every quantity reported above. The search engine is deterministic: tie-breaking in the open list is by insertion order, and patterns are a deterministic function of the task and the budget, so repeated runs of a configuration return identical metrics. The four instances are generated from a fixed seed and are committed.

References

- [1] B. Bonet and H. Geffner. Planning as heuristic search. *Artificial Intelligence*, 129(1–2):5–33, 2001.
- [2] J. C. Culberson and J. Schaeffer. Pattern databases. *Computational Intelligence*, 14(3):318–334, 1998.
- [3] S. Edelkamp. Planning with pattern databases. In *Proceedings of the 6th European Conference on Planning (ECP)*, 2001.
- [4] P. E. Hart, N. J. Nilsson and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE*

Transactions on Systems Science and Cybernetics, 4(2):100–107, 1968.

- [5] P. Haslum and H. Geffner. Admissible heuristics for optimal planning. In *Proceedings of the 5th International Conference on Artificial Intelligence Planning and Scheduling (AIPS)*, 2000.
- [6] P. Haslum, A. Botea, M. Helmert, B. Bonet and S. Koenig. Domain-independent construction of pattern database heuristics for cost-optimal planning. In *Proceedings of the 22nd AAAI Conference on Artificial Intelligence*, 2007.
- [7] M. Helmert and C. Domshlak. Landmarks, critical paths and abstractions: what’s the difference anyway? In *Proceedings of the 19th International Conference on Automated Planning and Scheduling (ICAPS)*, 2009.
- [8] E. Karpas and C. Domshlak. Cost-optimal planning with landmarks. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI)*, 2009.
- [9] M. Katz and C. Domshlak. Optimal additive composition of abstraction-based admissible heuristics. In *Proceedings of the 18th International Conference on Automated Planning and Scheduling (ICAPS)*, 2008.
- [10] F. Pommerening, G. Röger, M. Helmert and B. Bonet. LP-based heuristics for cost-optimal planning. In *Proceedings of the 24th International Conference on Automated Planning and Scheduling (ICAPS)*, 2014.
- [11] F. Pommerening, M. Helmert, G. Röger and J. Seipp. From non-negative to general operator cost partitioning. In *Proceedings of the 29th AAAI Conference on Artificial Intelligence*, 2015.
- [12] J. Seipp and M. Helmert. Diverse and additive Cartesian abstraction heuristics. In *Proceedings of the 24th International Conference on Automated Planning and Scheduling (ICAPS)*, 2014.
- [13] J. Seipp, T. Keller and M. Helmert. Narrowing the gap between saturated and optimal cost partitioning for classical planning. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence*, 2017.
- [14] J. Seipp and M. Helmert. Counterexample-guided Cartesian abstraction refinement for classical planning. *Journal of Artificial Intelligence Research*, 62:535–577, 2018.
- [15] J. Seipp, T. Keller and M. Helmert. Saturated cost partitioning for optimal classical planning. *Journal of Artificial Intelligence Research*, 67:129–167, 2020.
- [16] J. Slaney and S. Thiébaux. Blocks world revisited. *Artificial Intelligence*, 125(1–2):119–153, 2001.
- [17] F. Yang, J. Culberson, R. Holte, U. Zahavi and A. Felner. A general theory of additive state space abstractions. *Journal of Artificial Intelligence Research*, 32:631–662, 2008.